Authentication: tables and data flow

As of 2026-10-19. Source: `backend/src/modules/auth/` and `backend/prisma/schema.prisma`.

Authentication touches **12 tables**:

- **7 are used by the code today:** `usr_user`, `usr_user_profile`, `auth_otp`, `auth_session`, `auth_password_history`, `auth_login_log`, and `player`, which gets a row on first use.
- **5 exist in the schema but nothing writes to them yet:** `auth_device`, `auth_password_reset`, `auth_account_verification`, `usr_role`, `usr_user_role`.

There are also two non-table stores: a cache that skips the session check for 60 seconds, and the token saved on the phone.

Tables in use

1. `usr_user`: the account

Column	Meaning
`id` (uuid)	User id; it goes into the token's `sub` claim
`username`	Unique, compared case-insensitively
`phone_country_code` + `phone_number`	Unique as a pair (`uq_usr_user_phone`); required
`email`	Unique, optional
`password_hash`	argon2id hash (OWASP settings: 19 MiB memory, 2 iterations; `password.ts`)
`status`	1 = active, 2 = suspended, 3 = deleted
`is_phone_verified`, `is_email_verified`	Flags; once set they stay set
`last_login_at`, `created_at`, `updated_at`	

2. `usr_user_profile`: display name

`user_id` (unique, references `usr_user`), `display_name`, `profile_photo`, `city_id`, `bio`. Auth only writes `display_name`, at sign-up. Every user lookup joins this table so `fullName` comes back with the user.

3. `auth_otp`: SMS codes

Column	Meaning
`user_id`	Null at sign-up, because the account doesn't exist yet
`phone_country_code`, `phone_number`	The phone the code was sent to
`otp_hash`	HMAC-SHA256 of `purpose:+phone:code`; the plain code is never stored
`purpose`	1 = registration, 2 = phone login, 3 = password reset
`attempt_count`, `max_attempts` (5)	Every wrong guess counts
`expires_at`	10 minutes after sending
`verified_at`	Set when the right code is entered, which uses the code up

4. `auth_session`: one row per signed-in device

Column	Meaning
`id` (uuid)	Goes into the access token's `sid` claim, and is the first half of the refresh token
`refresh_token_hash`	SHA-256 of the refresh secret; replaced every time the token is refreshed
`expires_at`	30 days, pushed out again on each refresh
`ip_address`, `user_agent`, `last_used_at`	Who and where
`revoked_at`	Set on logout, password reset, or reuse of an old refresh token
`device_id`	Exists but is never filled

5. `auth_password_history`

`user_id`, `password_hash`, `created_at`. A row is added at sign-up and on every password reset. **Nothing reads it yet**, so "don't reuse an old password" isn't enforced.

6. `auth_login_log`: audit trail

Column	Meaning
`user_id`	Null when the account wasn't found
`username`, or `phone_country_code` + `phone_number`	What the person typed
`login_method`	1 = password, 2 = OTP, 3 = Google, 4 = Apple
`action`	1 = login, 2 = logout
`status`	1 = success, 2 = failed
`failure_reason`	`wrong_password`, `unknown_account`, `invalid_code`, or an account-disabled reason
`ip_address`, `user_agent`	
`device_id`	Never filled

Writing to this log never fails a request.

7. `player` (new core): the user's player identity

This isn't written by auth itself. The first time a signed-in user opens their profile, joins a tournament or calls `/players/me`, `ensureForUser` creates the user's `player` row. `claimGuests` (`POST /players/claim`) attaches guest players to the account.

Tables in the schema that nothing uses yet

Table	Intended for	Status
`auth_device`	Device id, OS, app version, `push_token`	Not written. It's needed for push notifications and to fill `auth_session.device_id`.
`auth_password_reset`	Reset links by token	Not used; password reset uses `auth_otp` with purpose 3 instead.
`auth_account_verification`	Email or other verification	Not used; there's no email verification flow.
`usr_role`, `usr_user_role`	Roles such as admin	Not used. Tournament permissions come from `TournamentStaff` instead.

There is no Google/Apple identity table. That's why social sign-in only works for an **existing** account whose email the provider has verified.

Other stores

- **Session cache** (Redis, or memory when Redis isn't set up):

    - `auth:session:<id>` means "this session was checked and is active". It lasts 60 seconds.
    - `auth:user-revoked:<userId>` is set on password reset, so every cached session of that user stops counting.
- **On the phone:** `expo-secure-store` holds the access token, the refresh token and their expiry times. It uses the Keychain on iOS and the Keystore on Android.

How data moves through each flow

Sign up (`/register/start`, then `/register/complete`)

1. **Start:** the server checks `usr_user` for a taken username, email or phone. It expires any live registration code for that phone in `auth_otp`, inserts a new `auth_otp` row (purpose 1, `user_id` null), and sends the SMS.
2. **Complete:** the server checks availability again, then verifies the code. It locks the newest live code row (`FOR UPDATE`), increments `attempt_count`, and sets `verified_at` if the code is right.
3. In one transaction it inserts:
    - `usr_user`, with `is_phone_verified` true and `last_login_at` set to now
    - `usr_user_profile` (`display_name`)
    - `auth_password_history`
    - `auth_session`
4. It returns the user and tokens.

If someone else takes the same username, email or phone at the same moment, the unique index rejects the insert and the response is `ACCOUNT_EXISTS`. Sign-up doesn't write to `auth_login_log`.

Password login (`/login`, using phone, email or username)

1. Look up the account in `usr_user` joined with the profile.
2. Check the password. When the account doesn't exist, it still checks against a dummy hash, so the response time doesn't reveal whether the account exists.
3. On failure: write a failed `auth_login_log` row and return `INVALID_CREDENTIALS`.
4. On success: check the account is active, update `usr_user.last_login_at`, write a successful `auth_login_log` row, insert an `auth_session` row, and return tokens.

Phone code login (`/phone/start`, then `/phone/verify`)

1. **Start:** only an active, registered number gets a code. The `auth_otp` row has purpose 2 and the user's id. The response is identical either way.
2. **Verify:** check the code against `auth_otp`. On success:
    - update `usr_user.last_login_at` and set `is_phone_verified`
    - write an `auth_login_log` row with method 2
    - insert an `auth_session` row

Forgot password (`/password/forgot`, then `/password/reset`)

1. **Forgot:** for an active account, add an `auth_otp` row with purpose 3, texted to the account's phone. The response never says whether the account exists.
2. **Reset:** verify the code. Then, in one transaction:
   - update `usr_user.password_hash`
   - insert an `auth_password_history` row
   - set `revoked_at` on **all** of the user's `auth_session` rows
3. After that, `auth:user-revoked` is set in the cache, so every device is signed out at once.

Google or Apple (`/google`, `/apple`)

1. The provider's token is verified, and the provider's email must be verified.
2. Look up the account in `usr_user` by email. If there's none: write a failed `auth_login_log` row and return the "social account not found" error.
3. If found:
   - set `is_email_verified` and `last_login_at`
   - write an `auth_login_log` row with method 3 or 4
   - insert an `auth_session` row

Every authenticated request (`Authorization: Bearer <JWT>`)

1. Verify the JWT. It lasts 15 minutes and carries `sub` (user id) and `sid` (session id).
2. If the cache says the session is active, the request goes through.
3. Otherwise, query `auth_session` joined with `usr_user`. The session must not be revoked or expired, and the user's status must be active. If that passes, cache the answer for 60 seconds; if not, return `SESSION_REVOKED`.

Refresh (`/refresh`, with the token `<sessionId>.<secret>`)

1. Lock the `auth_session` row and check it isn't revoked or expired.
2. Compare the hash of the secret with `refresh_token_hash`:
   - **Match:** check the user is still active, then write a new hash, a new `expires_at` and `last_used_at`. Return a new pair of tokens.
   - **No match:** someone is reusing an old refresh token, so the server sets `revoked_at`, clears the cache entry and returns `INVALID_REFRESH_TOKEN`.

Logout (`/logout`)

- If the refresh token matches, set `auth_session.revoked_at`, clear the cache entry and write an `auth_login_log` row with action 2.
- An unknown token is ignored.

`/me`

Reads `usr_user` joined with the profile.

Rate limits

These are kept in the cache, not in a table:

- 300 requests per IP per 15 minutes.
- Sending codes: 20 per IP per hour and 5 per phone or account per hour.
- Checking codes: 10 per phone or account per 15 minutes.
- Password login: 10 per account per 15 minutes.

Gaps

- `auth_device` is never written, so `device_id` is empty in both `auth_session` and `auth_login_log`.
- Nothing reads `auth_password_history`.
- `auth_password_reset`, `auth_account_verification`, `usr_role` and `usr_user_role` aren't used.
- Expired `auth_otp` rows and old `auth_session` rows are never deleted, so they build up over time.